А теперь используй все что мы с тобой здесь наработал все математические модели все графики и анимации все проверки на системах и нейросетям сформируй единый пито код модели назови его и если необходимо сформируй дополнительные файлы

Ниже — единый проект **Responsibility Response Model (RRM)**. Он объединяет всё, что мы построили: базовую модель, коэффициент задержки, временную поддержку, проверку на человеке, финансах и нейросетях, графики и MP4/GIF-анимацию.

Структура проекта

Создай на рабочем столе папку:

```
ResponsibilityResponseModel/
├── universal_responsibility_model.py
├── requirements.txt
└── README.md
```

Такая структура с `requirements.txt` и `README.md` — стандартная практика для воспроизводимых Python-проектов. [1] [2] [3]

Файл 1: universal_responsibility_model.py

Сохрани этот код как `universal_responsibility_model.py`.

```
# -*- coding: utf-8 -*-
"""
Responsibility Response Model (RRM)
====================================

Универсальная модель ответственности:

V(T) = B(T) - R(T) + ∫ D(t) * S(t) dt

Где:
B(T) — ожидаемая польза решения
R(T) — ожидаемый вред или риск
S(t) — поддержка, компенсация, исправление
D(t) — коэффициент задержки помощи
```

```
T    — период ответственности

D(t) = exp(-lambda * t)

Модель применяется к:
- человеку;
- обществу;
- финансовой системе;
- нейросети;
- любой системе, создающей риск и обязанной реагировать.
"""

import importlib.util
import os
import shutil
import subprocess
import sys

REQUIRED_PACKAGES = ["numpy", "matplotlib", "pillow"]


# ----------------------------
# Проверка окружения
# ----------------------------

def check_python_packages():
    missing = []

    for package in REQUIRED_PACKAGES:
        spec = importlib.util.find_spec(package)

        if spec is None:
            missing.append(package)
            print(f"[НЕТ] {package} не установлен.")
        else:
            print(f"[OK] {package} установлен.")

    if missing:
        answer = input(
            f"\nДля работы нужны: {', '.join(missing)}.\n"
            "Установить их сейчас через pip? [y/n]: "
        ).strip().lower()

        if answer in ("y", "yes", "д", "да"):
            for package in missing:
                print(f"Устанавливаю {package}...")
                subprocess.check_call(
                    [sys.executable, "-m", "pip", "install", package]
                )
        else:
            print("\nУстановка отменена. Скрипт завершён.")
            sys.exit(1)

    for package in REQUIRED_PACKAGES:
        if importlib.util.find_spec(package) is None:
            print(f"Ошибка: {package} не удалось установить.")
```

```
            sys.exit(1)

    print("\nPython-библиотеки готовы.\n")


def check_ffmpeg():
    ffmpeg_path = shutil.which("ffmpeg")

    if ffmpeg_path is None:
        print("[НЕТ] ffmpeg не найден.")
        print("MP4 требует ffmpeg. Будет создан GIF.")
        return False

    print(f"[OK] ffmpeg найден: {ffmpeg_path}")
    return True


check_python_packages()
FFMPEG_AVAILABLE = check_ffmpeg()

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, FFMpegWriter, PillowWriter


# ----------------------------
# Ядро модели
# ----------------------------

def delay_coefficient(t, lam=0.15, critical_time=1.0):
    """
    Коэффициент задержки помощи.

    D(t) = 1, если t <= critical_time
    D(t) = exp(-lambda * (t - critical_time)), если t > critical_time
    """
    t = np.asarray(t, dtype=float)

    return np.where(
        t <= critical_time,
        1.0,
        np.exp(-lam * (t - critical_time))
    )


def model_value(B, R, S, lam, critical_time=1.0, T=20.0, steps=1000):
    """
    V(T) = B - R + ∫ D(t) * S(t) dt
    """
    t = np.linspace(0, T, steps)
    dt = t[^1] - t[^0]

    D = delay_coefficient(t, lam, critical_time)
    support_integral = np.sum(D * S * dt)

    return B - R + support_integral
```

```
def support_function(t, base_support=60.0, decay=0.03):
    """
    S(t) — непрерывная поддержка во времени.
    """
    t = np.asarray(t, dtype=float)
    return base_support * np.exp(-decay * t)


# ----------------------------
# Сценарии
# ----------------------------

SCENARIOS = {
    "Человек-спасатель": {
        "B": 100.0,
        "R": 80.0,
        "S_base": 60.0,
        "S_decay": 0.03,
        "lambda": 0.20,
        "critical_time": 1.0
    },
    "Финансовый кризис": {
        "B": 100.0,
        "R": 90.0,
        "S_base": 70.0,
        "S_decay": 0.05,
        "lambda": 0.15,
        "critical_time": 2.0
    },
    "Нейросеть": {
        "B": 100.0,
        "R": 70.0,
        "S_base": 55.0,
        "S_decay": 0.08,
        "lambda": 0.30,
        "critical_time": 0.5
    }
}


def evaluate_scenarios(T=20.0):
    results = {}

    for name, params in SCENARIOS.items():
        t = np.linspace(0, T, 1000)
        S = support_function(
            t,
            base_support=params["S_base"],
            decay=params["S_decay"]
        )

        value = model_value(
            B=params["B"],
            R=params["R"],
```

```
            S=S,
            lam=params["lambda"],
            critical_time=params["critical_time"],
            T=T
        )

        results[name] = value

    return results


# ----------------------------
# График 1: задержка помощи
# ----------------------------

def plot_delay_curves():
    t = np.linspace(0, 20, 500)

    fast = delay_coefficient(t, lam=0.05)
    medium = delay_coefficient(t, lam=0.15)
    slow = delay_coefficient(t, lam=0.30)

    plt.figure(figsize=(10, 6))

    plt.plot(t, fast, label="Быстрая система: λ = 0.05", linewidth=2.5)
    plt.plot(t, medium, label="Средняя система: λ = 0.15", linewidth=2.5)
    plt.plot(t, slow, label="Медленная система: λ = 0.30", linewidth=2.5)

    plt.axvline(1, color="red", linestyle="--", label="Критическое время")

    plt.title("Коэффициент задержки помощи D(t)")
    plt.xlabel("Время задержки t")
    plt.ylabel("Эффективность помощи D(t)")
    plt.grid(True, alpha=0.3)
    plt.legend()

    plt.tight_layout()
    plt.savefig("1_delay_coefficient.png", dpi=160)
    plt.close()


# ----------------------------
# График 2: сравнение систем
# ----------------------------

def plot_scenario_comparison():
    results = evaluate_scenarios()

    names = list(results.keys())
    values = list(results.values())

    plt.figure(figsize=(10, 6))

    bars = plt.bar(names, values, color=["#2e86de", "#e67e22", "#8e44ad"])

    plt.axhline(0, color="black", linewidth=1)
```

```
    plt.title("Итоговая ценность системы V(T)")
    plt.ylabel("V(T)")
    plt.grid(axis="y", alpha=0.3)

    for bar, value in zip(bars, values):
        plt.text(
            bar.get_x() + bar.get_width() / 2,
            value + 1,
            f"{value:.1f}",
            ha="center",
            fontsize=11
        )

    plt.tight_layout()
    plt.savefig("2_system_comparison.png", dpi=160)
    plt.close()


# ----------------------------
# График 3: влияние скорости реакции
# ----------------------------

def plot_reaction_speed():
    lambdas = np.linspace(0.01, 0.60, 200)
    values = []

    for lam in lambdas:
        t = np.linspace(0, 20, 1000)
        S = support_function(t, base_support=60.0, decay=0.03)

        value = model_value(
            B=100,
            R=80,
            S=S,
            lam=lam,
            critical_time=1.0,
            T=20.0
        )

        values.append(value)

    plt.figure(figsize=(10, 6))
    plt.plot(lambdas, values, color="#c0392b", linewidth=3)
    plt.axhline(0, color="black", linestyle="--")

    plt.title("Чем медленнее реакция, тем ниже V(T)")
    plt.xlabel("Скорость потери эффективности помощи λ")
    plt.ylabel("Итоговая ценность V(T)")
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig("3_reaction_speed.png", dpi=160)
    plt.close()


# ----------------------------
```

```
# График 4: принцип модели
# ----------------------------

def plot_model_principle():
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.axis("off")

    text = (
        "RESPONSIBILITY RESPONSE MODEL\n\n"
        "V(T) = B(T) - R(T) + ∫ D(t) S(t) dt\n\n"
        "B(T) — ожидаемая польза\n"
        "R(T) — ожидаемый вред\n"
        "S(t) — поддержка и исправление\n"
        "D(t) — коэффициент задержки\n"
        "T — период ответственности\n\n"
        "Главный принцип:\n"
        "Чем позже помощь, тем меньше она искупает вред.\n\n"
        "Человек: спасение — риск — поддержка.\n"
        "Государство: кризис — защита — восстановление.\n"
        "Банк: кризис — компенсация — доверие.\n"
        "Нейросеть: ошибка — реакция — исправление."
    )

    ax.text(
        0.5, 0.5, text,
        ha="center", va="center",
        fontsize=13,
        family="monospace"
    )

    plt.tight_layout()
    plt.savefig("4_model_principle.png", dpi=160)
    plt.close()


# ----------------------------
# Анимация
# ----------------------------

def create_animation(export_mp4=True):
    T_MAX = 20.0
    FRAMES = 120
    LAMBDA = 0.18
    CRITICAL_TIME = 1.0

    t_values = np.linspace(0, T_MAX, 500)

    B = 100.0
    R = 80.0
    S_base = 60.0
    S_decay = 0.03

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

    line_D, = ax1.plot(
        [], [],
```

```
        color="#c0392b",
        linewidth=3,
        label="Эффективность помощи D(t)"
    )

    ax1.set_xlim(0, T_MAX)
    ax1.set_ylim(0, 1.1)
    ax1.set_xlabel("Время задержки t")
    ax1.set_ylabel("D(t)")
    ax1.set_title("Задержка помощи")
    ax1.grid(True, alpha=0.3)
    ax1.legend(loc="upper right")

    line_V, = ax2.plot(
        [], [],
        color="#2e86de",
        linewidth=3,
        label="Итоговая ценность V(T)"
    )

    ax2.set_xlim(0, T_MAX)
    ax2.set_ylim(-20, 100)
    ax2.set_xlabel("Время задержки t")
    ax2.set_ylabel("V(T)")
    ax2.set_title("Итоговая ценность системы")
    ax2.grid(True, alpha=0.3)
    ax2.legend(loc="upper right")

    text = ax2.text(
        0.02, 0.90,
        "",
        transform=ax2.transAxes,
        fontsize=11,
        family="monospace",
        bbox=dict(facecolor="white", alpha=0.8)
    )

    def update(frame):
        current_t = T_MAX * frame / (FRAMES - 1)

        D_current = delay_coefficient(
            np.array([current_t]),
            lam=LAMBDA,
            critical_time=CRITICAL_TIME
        )[^0]

        S_current = support_function(
            np.array([current_t]),
            base_support=S_base,
            decay=S_decay
        )[^0]

        V_current = model_value(
            B=B,
            R=R,
            S=S_current,
```

```
            lam=LAMBDA,
            critical_time=CRITICAL_TIME,
            T=current_t
        )

        t_plot = t_values[t_values <= current_t]

        D_plot = delay_coefficient(
            t_plot,
            lam=LAMBDA,
            critical_time=CRITICAL_TIME
        )

        V_plot = []
        for time_point in t_plot:
            S_point = support_function(
                np.array([time_point]),
                base_support=S_base,
                decay=S_decay
            )[^0]

            value = model_value(
                B=B,
                R=R,
                S=S_point,
                lam=LAMBDA,
                critical_time=CRITICAL_TIME,
                T=time_point
            )

            V_plot.append(value)

        line_D.set_data(t_plot, D_plot)
        line_V.set_data(t_plot, V_plot)

        text.set_text(
            f"t = {current_t:.2f}\n"
            f"D(t) = {D_current:.3f}\n"
            f"S(t) = {S_current:.2f}\n"
            f"V(T) = {V_current:.2f}"
        )

        return line_D, line_V, text

    ani = FuncAnimation(
        fig,
        update,
        frames=FRAMES,
        interval=60,
        blit=False,
        repeat=True
    )

    if export_mp4 and FFMPEG_AVAILABLE:
        output_file = "model_delay_animation.mp4"
```

```
        writer = FFMpegWriter(
            fps=30,
            bitrate=2400,
            metadata={
                "title": "Responsibility Response Model",
                "artist": "RRM"
            }
        )

        ani.save(
            output_file,
            writer=writer,
            dpi=150
        )

        print(f"Создан MP4: {output_file}")

    else:
        output_file = "model_delay_animation.gif"

        writer = PillowWriter(fps=15)
        ani.save(
            output_file,
            writer=writer,
            dpi=120
        )

        print(f"Создан GIF: {output_file}")


# ----------------------------
# Главная функция
# ----------------------------

def main():
    print("Запуск Responsibility Response Model...\n")

    plot_delay_curves()
    print("Создан график: 1_delay_coefficient.png")

    plot_scenario_comparison()
    print("Создан график: 2_system_comparison.png")

    plot_reaction_speed()
    print("Создан график: 3_reaction_speed.png")

    plot_model_principle()
    print("Создан график: 4_model_principle.png")

    create_animation(export_mp4=True)

    print("\nВсе файлы созданы успешно.")


if __name__ == "__main__":
    main()
```

Файл 2: requirements.txt

Сохрани как `requirements.txt`:

```
numpy
matplotlib
pillow
```

Файл 3: README.md

Сохрани как `README.md`:

```
# Responsibility Response Model (RRM)

Универсальная модель ответственности:

V(T) = B(T) - R(T) + ∫ D(t) * S(t) dt

Где:
- B(T) — ожидаемая польза
- R(T) — ожидаемый вред
- S(t) — поддержка, компенсация, исправление
- D(t) — коэффициент задержки помощи
- T — период ответственности

D(t) = exp(-lambda * t)

## Что моделирует

- Человека, который спасает другого и получает травму.
- Государство, которое реагирует на кризис.
- Финансовую систему во время кризиса.
- Нейросеть, которая ошибается и должна исправлять вред.
- Любую систему, где задержка помощи увеличивает ущерб.

## Как запустить

1. Установи Python.
2. Открой терминал в папке проекта.
3. Выполни:

```bash
python universal_responsibility_model.py
```

Скрипт сам проверит наличие библиотек и предложит установить недостающие.

Что создаётся

- 1_delay_coefficient.png
- 2_system_comparison.png
- 3_reaction_speed.png
```

- 4_model_principle.png
- model_delay_animation.mp4, если установлен ffmpeg
- model_delay_animation.gif, если ffmpeg не установлен

Главная идея

Чем позже помощь, тем меньше она искупает вред.

Ответственность — это не разовый поступок,
а непрерывная реакция во времени.

````
## Как запустить

1. Создай папку `ResponsibilityResponseModel` на рабочем столе.
2. Помести туда три файла:
   - `universal_responsibility_model.py`
   - `requirements.txt`
   - `README.md`
3. Открой терминал в этой папке.
4. Выполни:

```bash
python universal_responsibility_model.py
````

Что создаст скрипт

После запуска появятся:

```
1_delay_coefficient.png
2_system_comparison.png
3_reaction_speed.png
4_model_principle.png
model_delay_animation.mp4
```

Если `ffmpeg` не установлен:

```
model_delay_animation.gif
```

Главная формула проекта

$$\boxed{V(T) = B(T) - R(T) + \int_0^T e^{-\lambda t} S(t) dt}$$
```

Универсальный принцип

> Любая система ответственна не только за решение, но и за скорость и качеств

Финальный вывод

Этот единый код объединяет:

- базовую модель;
- коэффициент задержки;
- временную поддержку;
- проверку на человеке;
- проверку на финансах;
- проверку на нейросетях;
- графики;
- анимацию;
- экспорт в MP4;
- резервный экспорт в GIF.

Название проекта:

Responsibility Response Model (RRM)

Смысл модели:

> Настоящая ответственность — это не момент решения, а непрерывная и своевр

Matplotlib поддерживает и GIF через `PillowWriter`, и MP4 через `FFMpegWriter`; `FFMpegWriter` использует кодек `h264` по умолчанию. [4] [5] [6]

⁂

1. https://arxiv.org/html/2606.14924
2. https://docs.python-guide.org/writing/structure/
3. https://learn.colinkim.dev/courses/python/project-structure-and-organization/
4. https://matplotlib.org/3.7.1/tutorials/introductory/animation_tutorial.html
5. https://matplotlib.org/3.10.9/api/animation_api.html
6. https://matplotlib.org/3.7.0/api/_as_gen/matplotlib.animation.FFMpegWriter.html
7. https://github.com/matplotlib/matplotlib.github.com/blob/main/3.11.0/api/_as_gen/matplotlib.animation.FFMpegWriter.html
8. https://github.com/python/cpython/blob/3.8/Doc/library/importlib.rst
9. https://github.com/iamwaqarjaved/python-packaging-guide/blob/main/docs/01-project-structure.md
10. https://github.com/iamwaqarjaved/python-packaging-guide/blob/main/README.md

11. https://docs.python.org/3/library/importlib.html
12. https://matplotlib.org/stable/api/animation_api.html
13. https://matplotlib.org/3.9.0/users/explain/animations/animations.html
14. https://www.resus.org.uk/professional-library/2025-resuscitation-guidelines/ethics-guidelines
15. https://matplotlib.org/3.2.1/api/_as_gen/matplotlib.animation.PillowWriter.html
16. https://matplotlib.org/stable/_sources/api/animation_api.rst.txt